



UNIVERSIDADE FEDERAL DE PERNAMBUCO

CENTRO DE INFORMÁTICA

ESTRUTURAS DE DADOS ORIENTADAS A OBJETOS:

RELATÓRIO DO PROJETO TEAR

Professor responsável - Francisco Paulo Magalhães Simões

Discentes responsáveis pelo projeto:

Maria Gabriela Heleno da Silva <mghs>

David Everton da Silva Sales <dess2>

Heloísa Maria Santos Leite <hmsl>

Kailani Yoná da Silva Souza <kyss>

Recife, 2026.

1. TÍTULO & DESCRIÇÃO GERAL

O **Tear** é uma aplicação de gestão e precificação para a produção artesanal, como peças de crochê e amigurumi. O sistema consolida estoque, compras, orçamentos, acompanhamento de pedidos e gestão financeira com isolamento de dados por conta de usuário. O projeto foi desenvolvido sob a arquitetura de back-end em C++17 onde concentram-se as regras de negócio e persistência via SQLite3 e uma interface gráfica web responsiva desenvolvida em React.

Os principais objetivos, do ponto de vista acadêmico, foram: Aplicação prática de orientação a objetos: encapsulamento, herança, polimorfismo, abstração e gestão consciente de memória (ponteiros/referências); Mapeamento e execução de operações de CRUD com persistência em banco de dados relacional; Arquitetura desacoplada client-server via comunicação HTTP/REST com JSON; Organização de fluxo de trabalho colaborativo em equipe utilizando Git/GitHub.

2. EQUIPE E DIVISÃO DE MÓDULOS

Integrante	Módulos e Responsabilidades
Maria Gabriela	Estoque e Compras
David Everton	Clientes, Orçamentos e Pedidos (Kanban)
Heloísa Maria	Financeiro, Painel inicial, contas de usuário e banco de dados
Kailani Yoná	Projetos e Produção

Cada integrante ficou responsável pela camada em C++ e pelas telas em React correspondentes. A integração foi realizada por pull requests antes do merge na branch *develop* do repositório. No pull requests definimos um modelo de comentário que informa a descrição da mudança, o que foi testado e, quando necessário, capturas de tela.

3. COMO COMPILAR E EXECUTAR

São necessários um compilador C++17 (g++ ou MSVC), CMake 3.10 ou superior e Node.js. O SQLite já vem no repositório, então não há instalação de banco.

- Clonar o repositório: `git clone https://github.com/davdsales/tear.git`;
- Compilar o back-end, dentro da pasta backend: `cmake -B build` e depois `cmake --build build`;

- Executar o servidor a partir da pasta backend; ele responde em *http://localhost:8080*;
- Em outro terminal, dentro da pasta frontend: *npm install* e depois *npm run dev*;
- Abrir no navegador o endereço indicado pelo Vite, criar uma conta e entrar.

A compilação e o fluxo principal da API (cadastro de conta, material, compra confirmada, bloqueio por estoque insuficiente, criação de pedido e isolamento entre contas) foram verificados durante a revisão deste relatório.

4. REGRAS DE NEGÓCIO E FLUXO DO SISTEMA

A lógica do sistema organiza em **quatro** domínios principais:

4.1 Autenticação e sessão

O acesso exige autenticação (e-mail e senha). As requisições HTTP identificam a conta ativa através do cabeçalho da requisição. O servidor isola os dados em memória e no banco de dados, fazendo com que as consultas filtrem os registros pertencentes ao ID do usuário identificado na requisição.

4.2 Estoque e compras

- *Tipagem de materiais:* Materiais categorizam-se em Fio, Tecido e Aviamento, mantendo atributos comuns (nome, unidade, custo unitário e estoque mínimo) e específicos (como metragem para fios ou largura para tecidos);
- *Cálculo do saldo:* O saldo em estoque não é editável diretamente, pois deriva da soma algébrica do histórico de movimentações (entradas, consumos e ajustes);
- *Consistência e integridade:* A baixa de material valida a disponibilidade atual. Não são permitidos saídas superiores ao saldo existente nem exclusão de materiais que possuam histórico de movimentações vinculado;
- *Compras:* O registro de compra consolida itens e fornecedores. Ao confirmar uma compra, o sistema valida os materiais envolvidos e registra automaticamente as respectivas entradas no estoque. Após a confirmação, a compra passa a ser considerada concluída pelo sistema e não pode ser confirmada novamente.

4.3 Orçamentos e Pedidos

- *Composição de Custo e Preço:* O orçamento consolida custos de materiais, mão de obra e custos adicionais, aplicando a margem de lucro e desconto configurados. O preço final é limitado a valores não-negativos.

- *Preservação de Histórico:* Ao converter um orçamento em pedido, os dados do orçamento são copiados para a entidade Pedido. Dessa forma, o pedido mantém os valores registrados no momento de sua criação.
- *Fluxo Kanban:* O registro de compra consolida itens e fornecedores. Ao confirmar uma compra, o sistema valida os materiais envolvidos e registra automaticamente as respectivas entradas no estoque. Após a confirmação, a compra passa a ser considerada concluída pelo sistema e não pode ser confirmada novamente.

4.4 Gestão financeira

Registra lançamentos categorizados entre Receita e Despesa. O painel sumariza o faturamento mensal, saldo a receber de pedidos ativos, alertas de estoque crítico e o balanço financeiro dos últimos seis meses.

5. DIAGRAMA DE CLASSES

Os diagramas a seguir seguem a notação UML: – indica atributo privado, # protegido e + público; “= 0” marca um método virtual puro; o triângulo vazio indica herança, o losango cheio indica composição (o lado do losango contém o outro objeto) e a linha tracejada indica dependência. Para facilitar a leitura, o modelo foi dividido em três figuras.

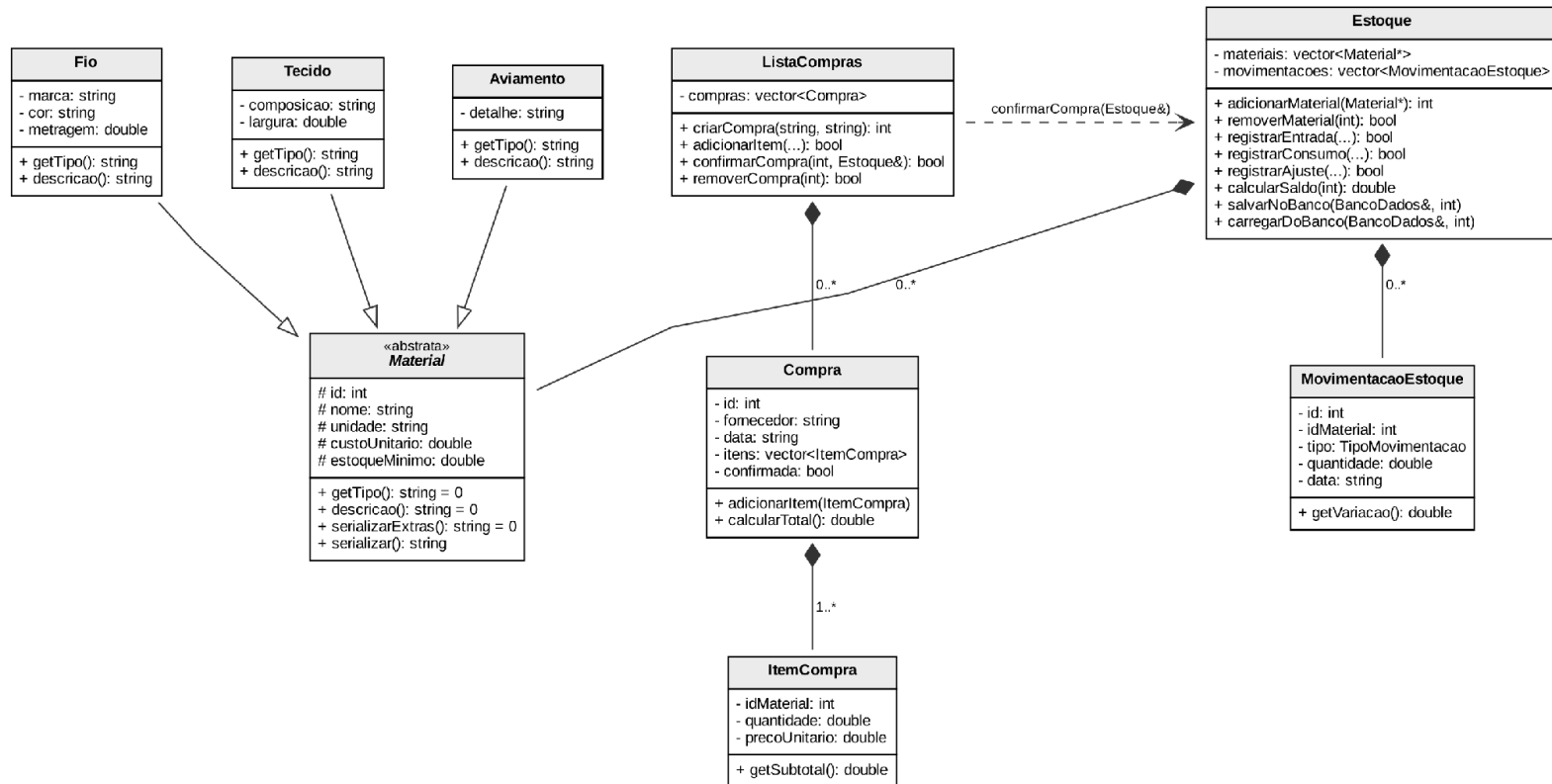


Figura 1 - Diagrama de classes do módulo de estoque e compras. A classe abstrata Material é especializada em Fio, Tecido e Aviamento. **Fonte:** Os autores, 2026.

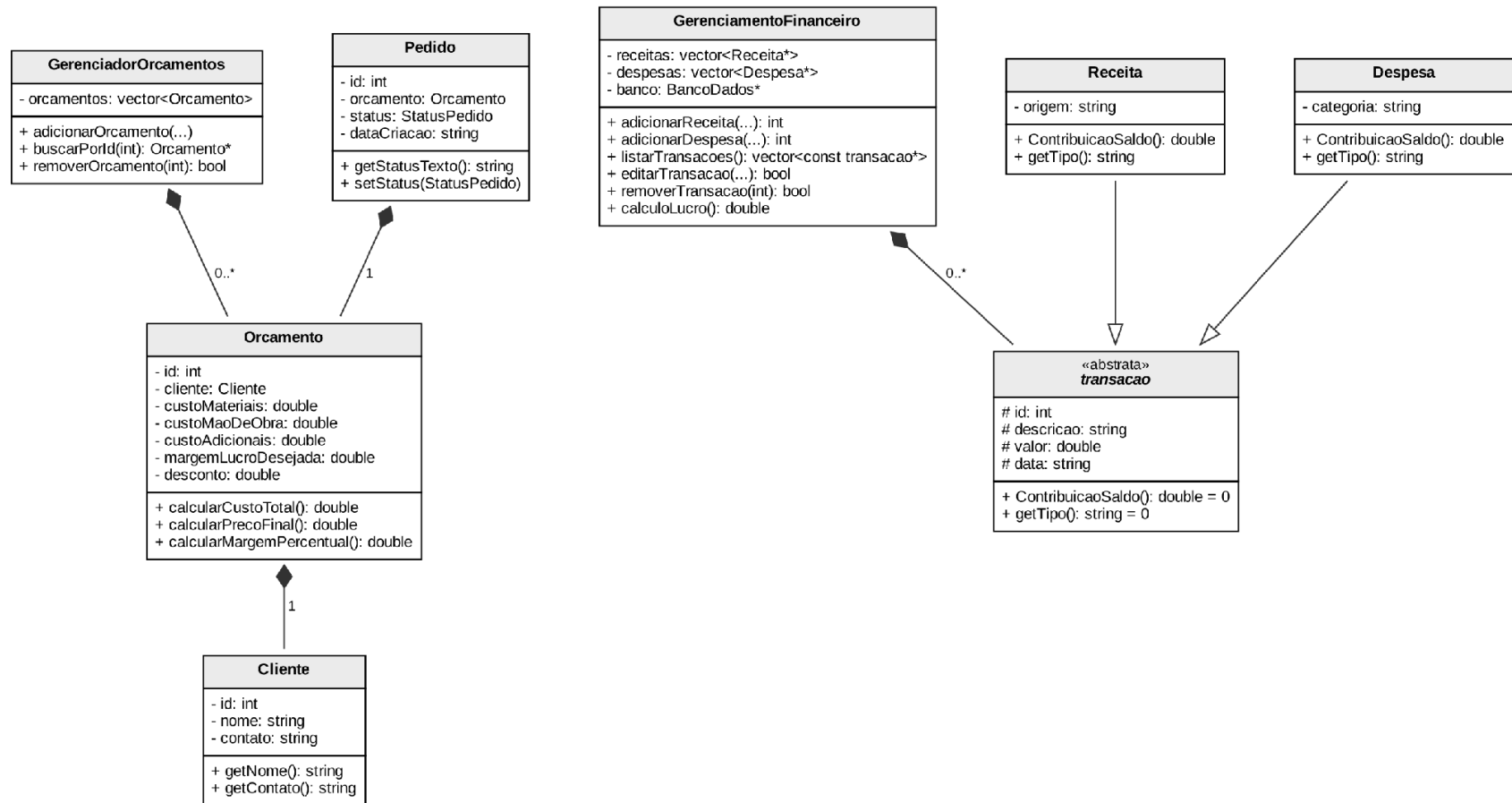


Figura 2 - Diagrama de classes dos módulos de orçamentos, pedidos e financeiro. A classe abstrata de transação é especializada em Receita e Despesa. **Fonte:** Os autores, 2026.

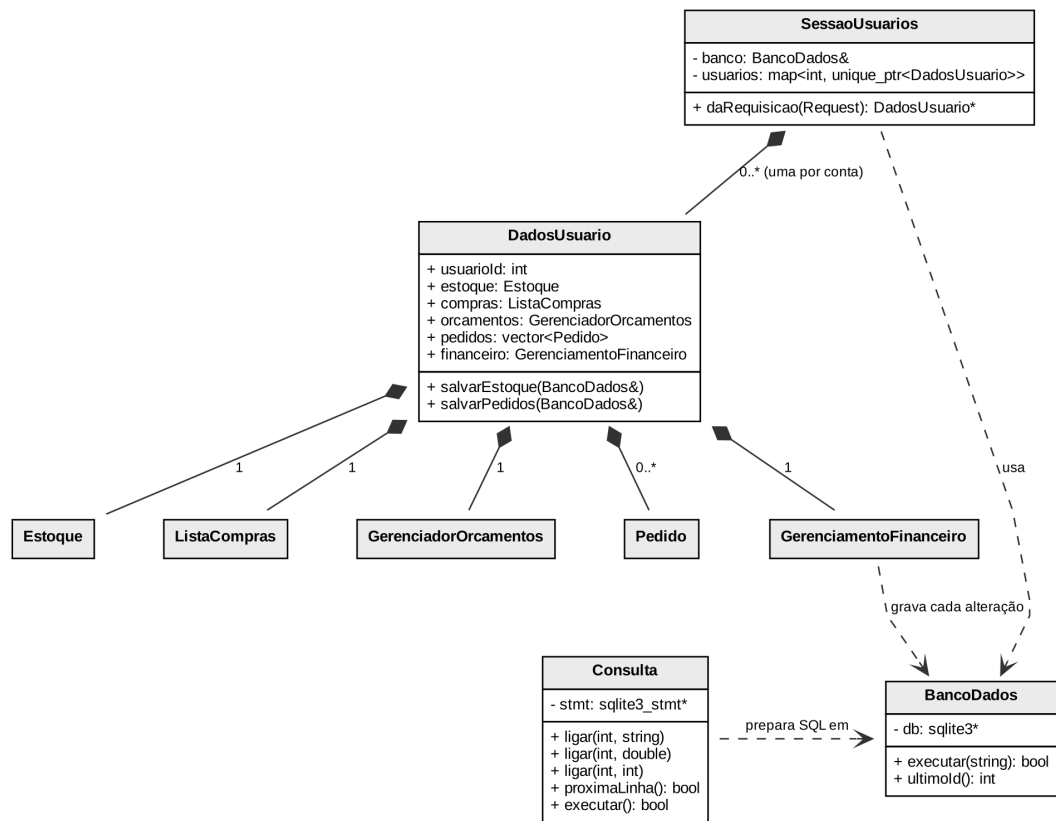


Figura 3 - Diagrama de classes da sessão de usuário e da camada de persistência. **Fonte:** Os autores, 2026.

Duas relações merecem destaque. O *Pedido* guarda uma **cópia** do *Orcamento*, e não um ponteiro para ele, para que os valores negociados fiquem preservados mesmo que o orçamento original mude. Já *ItemCompra* e *MovimentacaoEstoque* se associam a um material apenas pelo seu identificador (*idMaterial*), do mesmo modo que uma chave estrangeira no banco de dados.

6. ARQUITETURA DO PROJETO

Camada	Decisão	Justificativa
Aplicação - Back-end	Linguagem C++17	Padrão da disciplina; foco em orientação a objetos
Aplicação - Back-end	Biblioteca cpp-http lib	Servidor HTTP e rotas em um único arquivo de cabeçalho, sem instalação
Aplicação - Back-end	Biblioteca nlohmann/json	Converte dados entre objetos C++ e JSON de forma direta
Compilação	CMake	Compila o mesmo projeto no Windows e no Linux
Apresentação - Front-end	React 18, Vite e React Router	Interface gráfica com componentes reutilizáveis (bônus do enunciado)

Camada	Decisão	Justificativa
Persistência - Banco de Dados	SQLite 3	Banco em arquivo único, compilado junto com o programa, sem servidor separado

6.1 Visão Lógica

O sistema é estruturado em torno das principais entidades do domínio (ver tópico 5): Material e seus tipos, Movimentação de estoque, Compra, Orçamento, Cliente, Pedido e Transação (receita ou despesa). Todas pertencem a uma conta de usuário, representada pela classe *DadosUsuario*.

As principais regras de negócio, como cálculo de saldo, precificação e validações, ficam concentradas no back-end. A interface envia as ações do usuário por requisições HTTP, o servidor processa as regras nos objetos correspondentes e devolve os resultados em JSON. O sistema é organizado em dois módulos principais, back-end e front-end, que se comunicam por HTTP.

7. Mapeamento da API REST

Todos os dados ficam em um único arquivo SQLite, com oito tabelas criadas pelo próprio código C++ ao iniciar o servidor. Cada linha guarda o *usuario_id* da dona, e toda consulta é filtrada por ele.

Tabela	Classe responsável	O que guarda
usuarios	SessaoUsuarios	Nome, e-mail único e senha
materiais	Estoque (Fio, Tecido, Aviamento)	Dados comuns e colunas próprias de cada tipo
movimentacoes	Estoque (MovimentacaoEstoque)	Entrada, consumo ou ajuste, com data e observação
compras	ListaCompras (Compra)	Fornecedor, data e se a compra foi confirmada
itens_compra	ListaCompras (ItemCompra)	Material, quantidade e preço de cada item
orcamentos	GerenciadorOrcamentos (Orcamento)	Cliente, custos, percentual de lucro e desconto
pedidos	DadosUsuario (Pedido)	Orçamento de origem, status e data
transacoes	GerenciamentoFinanceiro (Receita, Despesa)	Tipo, descrição, valor, data, origem ou categoria

Os três tipos de material ficam na mesma tabela, numa estratégia conhecida como “uma tabela por hierarquia”: a coluna tipo indica qual subclasse deve ser recriada na leitura. O mesmo vale para receitas e despesas na tabela *transacoes*.

As operações de CRUD são expostas pela API conforme a tabela abaixo. Movimentações de estoque não são editadas nem apagadas, porque formam o histórico do saldo.

Recurso	Método HTTP	Rota	Descrição / Operação
Usuários	POST	/api/usuarios /api/login	Cadastro de usuário e autenticação de sessão.
Materiais	GET / POST / PUT / DELETE	/api/materiais	Gestão do catálogo de materiais.
Movimentações	GET / POST	/api/estoque/movimentacoes	Registro de entradas/saídas e consulta de histórico.
Compras	GET / POST / DELETE	/api/compras	Lançamento de ordens de compra e confirmação.
Orçamentos e pedidos	GET / POST / PUT	/api/pedidos	Acompanhamento e atualização de status no Kanban.
Transações	GET / POST / PUT	/api/financeiro/transacoes	Lançamento e edição de receitas e despesas

Além dessas, as rotas *GET /api/estoque*, *GET /api/dashboard* e *GET /api/financeiro/resumo* fornecem os resumos usados nas telas.

8. CONCEITOS DE ORIENTAÇÃO A OBJETOS

Os conceitos de POO foram utilizados da seguinte forma:

Classes e objetos: cada entidade do negócio é uma classe, e cada material, compra, orçamento ou pedido cadastrado é um objeto. O código está todo comentado em português, explicando as regras de cada método.

Encapsulamento e modificadores de acesso: os atributos são *private* ou *protected* e só mudam por métodos públicos que validam os dados. Os setters de *Orcamento*, por exemplo, não aceitam valores negativos, e o construtor usa esses setters para validar o objeto desde a criação:

```
void setCustoMateriais(double valor) { custoMateriais = std::max(0.0, valor); }
```

Abstração e herança: *Material* é uma classe abstrata com os dados comuns a todo material e métodos virtuais puros. Fio, Tecido e Aviamento herdam dela e acrescentam seus próprios atributos. O mesmo ocorre com *transacao*, especializada em *Receita* e *Despesa*.

```
class Material {
protected:
    int id;
    std::string nome;
    double custoUnitario;
public:
    virtual std::string getTipo() const = 0;
    virtual std::string descricao() const = 0;
};

class Fio : public Material {
private:
    std::string marca, cor;
    double metragem;
public:
    std::string getTipo() const override { return "FIO"; }
};
```

Polimorfismo por sobrescrita: o estoque guarda todos os materiais em um vetor de ponteiros para a classe base. Ao montar a resposta da API, o código chama *descricao()* sem saber o tipo do material, e cada subclasse responde do seu jeito. No financeiro, *ContribuicaoSaldo()* devolve o valor positivo para receitas e negativo para despesas.

```
std::vector<Material*> materiais;
...
{"descricao", m.descricao()}, // cada tipo descreve a si mesmo
```

Polimorfismo por sobrecarga: a classe *Consulta* tem três versões do método *ligar()*, uma para cada tipo de valor, e o compilador escolhe a correta. *GerenciamentoFinanceiro* faz o mesmo com *adicionarTransacao(Receita*)* e *adicionarTransacao(Despesa*)*.

```
void ligar(int posicao, const std::string& valor);
void ligar(int posicao, double valor);
void ligar(int posicao, int valor);
```

Ponteiros e referências: ponteiros para classes base permitem trabalhar com objetos de diferentes subclasses por meio do polimorfismo. Por exemplo, *std::vector<Material*>* permite armazenar diferentes tipos derivados de *Material* e chamar seus métodos virtuais por uma interface comum. Ponteiros também podem indicar ausência de resultado, como em *buscarPorId()*, que pode retornar *nullptr*, enquanto referências evitam cópias e permitem alterar diretamente um objeto existente, como ocorre na confirmação de uma compra:

```
bool confirmarCompra(int id, Estoque& estoque, const std::string& dataEntrada = "");
```

Composição e associação: *Compra* contém seus *ItemCompra*, *Orcamento* contém um *Cliente*, *Pedido* contém uma cópia do *Orcamento* e *DadosUsuario* contém um gerenciador de

cada módulo. Itens de compra e movimentações apenas se associam a um material pelo identificador.

Construtores, destrutores e cópia proibida: as classes utilizam construtores e listas de inicialização para definir corretamente o estado inicial dos objetos. Nas classes responsáveis por recursos ou objetos alocados dinamicamente, os destrutores realizam a liberação necessária. Quando a cópia poderia causar compartilhamento ou liberação incorreta desses recursos, o construtor de cópia é desabilitado com `= delete`.

Enumerações fortes: *enum class StatusPedido* e *enum class TipoMovimentacao* dão nome aos estados, sem números soltos no código.

9. CONCLUSÃO

O Tear mostra que os conceitos de orientação a objetos resolvem um problema concreto. A herança a partir de *Material* permite tratar fios, tecidos e aviamentos de forma uniforme, o encapsulamento protege regras como a de estoque insuficiente, e a composição organiza documentos formados por outros objetos. A equipe partiu de uma ideia de produto ampla e recortou o escopo para caber no prazo, mantendo o C++ como centro do projeto. O resultado é um back-end orientado a objetos, ligado a um banco SQLite com dados isolados por conta e consumido por uma interface gráfica em React.